COURSEWORK

LANCASTER UNIVERSITY

SCHOOL OF COMPUTING AND COMMUNICATIONS

# SCC.454: Large Scale Platforms for AI and Data Analysis

*Author:*
Dr Tharindu Ranasinghe

Date: January 15, 2026

# 1 Introduction

Your coursework is to complete a comprehensive Big Data Analytics project. This entails designing, implementing, and executing a complete data pipeline that encompasses database schema design, similarity computation, clustering analysis, and recommendation system development. Working in groups of up to five, you will demonstrate your understanding of distributed computing principles, scalable machine learning algorithms, and professional software engineering practices that we have explored throughout this module.

# 2 The Project

Your team is a group of machine learning engineers at Amazon. You have been tasked with building a comprehensive recommendation system to improve product discovery and customer experience. The business team would like to better understand customer purchasing behaviour, identify similar products and customers, segment the market through clustering analysis, and ultimately deliver personalised product recommendations.

The data science leadership has provided you with access to the Amazon Reviews 2023 dataset, a rich collection of customer reviews and product metadata. They have done a decent job in compiling the data; however, it will require careful preprocessing and schema design before analysis can begin.

Your team is expected to deliver a complete end-to-end solution that demonstrates your understanding of database design principles, scalable similarity computation, clustering techniques, and recommendation algorithms. The engineering managers will be reviewing not only your technical implementation but also your collaborative development practices through your GitLab repository. You are also required to produce a comprehensive technical report documenting your approach, findings, and recommendations for future improvements.

# 3 Data

The data you will be working on is the Amazon Reviews 2023 dataset, a large-scale collection compiled by McAuley Lab at UC San Diego. This is one of the most comprehensive e-commerce datasets publicly available, containing over 571 million customer reviews spanning 33 product categories and covering interactions from May 1996 to September 2023.

The dataset comprises two main components: user reviews and item metadata. User reviews include ratings (1-5 stars), review titles and text, helpfulness votes, timestamps at second-level precision, verified purchase status, and user identifiers. Item metadata provides product information, including titles, descriptions, bullet-point features, pricing, brand and store information, category hierarchies, images, and, importantly, co-purchase relationships (products frequently bought together).

A detailed schema describing the data attributes can be found at:
`https://amazon-reviews-2023.github.io/`.

# 4 Tasks

## 4.1 Task 1: Database System Design

### 4.1.1 Objective

Design, implement, and justify a scalable database system for storing and querying the Amazon Reviews 2023 dataset. This task focuses on selecting appropriate database technologies, designing suitable schemas or data models, and evaluating query performance on realistic workloads.

### 4.1.2 Context

Given the scale, heterogeneity, and semi-structured nature of the Amazon review and product metadata, your system should be designed with scalability, query efficiency, and flexibility in mind. Real-world e-commerce platforms must support multiple access patterns simultaneously: customer-facing product pages requiring fast response times, administrative dashboards requiring complex analytics, and recommendation engines demanding flexible querying.

### 4.1.3 Requirements

You are required to explore and evaluate **at least three different database systems**. You must justify your technology choices based on the characteristics of the data and the query patterns.
For each database system you evaluate, implement the schema/data model and measure query performance for the following queries:

1. **Product Information Retrieval:** Given a product ID (`parent_asin`), retrieve the product title and full product description.

2. **Recent Reviews:** Given a product ID (`parent_asin`), retrieve the most recent $N$ customer reviews, including rating, review title, review text, and timestamp.

3. **Keyword Search:** Given a keyword, return all products whose title or description contains the keyword.

4. **User Review History:** Given a user ID, retrieve all products reviewed by that user along with the corresponding ratings and review dates.

5. **Product Statistics:** Retrieve the average rating, rating distribution (count per star level), and total number of reviews for a given product.

#### 4.1.4 Deliverables

- **Schema Designs:** For each database system, provide schema diagrams or data model documentation showing your design choices.
- **Implementation:** Working code to load data and execute each query.
- **Performance Analysis:** A comparison table showing query execution times across all database systems for each query type. Include the experimental setup (hardware specifications, data volume, indexing strategies).
- **Justification:** A written analysis (500–750 words) discussing:
  - Why you selected each database technology
  - Trade-offs between the approaches (consistency, flexibility, scalability, query complexity)
  - Which database system you would recommend for a production e-commerce platform and why

### 4.2 Task 2: Similarity Computation

#### 4.2.1 Objective

Compute and analyse similarity relationships between entities in the Amazon Reviews 2023 dataset. This task focuses on measuring **product–product similarity** and **user–user similarity** using scalable algorithms and vector database technologies.

#### 4.2.2 Context

Similarity computation is fundamental to recommendation systems, search functionality, and clustering. However, naive pairwise comparison has $O(n^2)$ complexity, making it infeasible for datasets with millions of items. This task requires you to implement both exact and approximate similarity methods, understand the accuracy–performance trade-off, and evaluate different approaches to scalable similarity search.

#### 4.2.3 Requirements

**Part A: Product Similarity** Implement and compare **at least three different approaches** to computing product similarity. The approaches should include different feature vectors and vector databases.
For each approach, implement the following query:

- Given a product ID (`parent_asin`), retrieve the $N$ most similar products.

**Part B: User Similarity** Implement and compare **at least three different approaches** to computing user similarity. The approaches should include different feature vectors and vector databases.
or each approach, implement the following query:

- Given a user ID, retrieve the $N$ most similar users.

#### 4.2.4 Deliverables

- **Feature Engineering Pipeline:** Code showing how you construct feature vectors for products and users.
- **Similarity Implementations:** Working code for each similarity approach.
- **Query Results:** For 5 sample products and 5 sample users, show the top-10 similar items found by each method.
- **Performance Analysis:** Query execution times for different dataset sizes and parameter settings.
- **Quality Evaluation:** A discussion (400–600 words) analysing:
  - How "good" are the similarity results? Do the returned items make intuitive sense?
  - Which vector database would you recommend for a production e-commerce system and why? Consider factors such as scalability, query latency, indexing speed, and ease of integration.

### 4.3 Task 3: Clustering Analysis

#### 4.3.1 Objective

Apply clustering algorithms to segment products and customers in the Amazon Reviews dataset. This task requires you to discover meaningful groupings that could inform business decisions such as targeted marketing, inventory management, or personalised user experiences.

#### 4.3.2 Context

Clustering enables unsupervised discovery of natural groupings in data. In e-commerce, clustering can segment products into coherent groups for merchandising, identify customer personas for targeted marketing, or detect anomalous purchasing patterns. Effective clustering requires thoughtful feature engineering, appropriate algorithm selection, and rigorous evaluation.

#### 4.3.3 Requirements

**Part A: Product Clustering** Implement and compare **at least two different approaches** to clustering products into meaningful segments. The approaches should include different feature representations and clustering algorithms.
For each approach, provide:

- Cluster assignments for all products in your dataset.
- A profile of each cluster including size, defining characteristics, and representative examples.

**Part B: Customer Clustering** Implement and compare **at least two different approaches** to clustering customers into meaningful segments. The approaches should include different feature representations and clustering algorithms.
For each approach, provide:

- Cluster assignments for all customers in your dataset.
- A profile of each cluster including size, defining characteristics, and representative examples.

#### 4.3.4 Deliverables

- **Feature Engineering Pipeline:** Code showing how you construct feature vectors for products and customers.
- **Clustering Implementations:** Working code for each clustering approach applied to both products and customers.
- **Parameter Selection:** Evidence of how you determined optimal parameters (e.g., number of clusters) with appropriate justification.
- **Evaluation Metrics:** Quantitative evaluation of your clustering results using appropriate metrics.
- **Analysis:** A discussion (400–600 words) covering:
  - What do the discovered segments represent? Assign descriptive names to your clusters.
  - How do different clustering algorithms compare in terms of quality and interpretability?
  - What business applications could benefit from these customer and product segments?

## 4.4 Task 4: Recommendation System

### 4.4.1 Objective

Build, evaluate, and compare multiple recommendation approaches to predict user preferences and generate personalised product recommendations.

### 4.4.2 Context

Recommendation systems are a cornerstone of modern e-commerce, responsible for a significant portion of purchases on platforms like Amazon. This task requires you to implement and evaluate multiple recommendation approaches, handle practical challenges such as the cold-start problem, and demonstrate understanding of how different techniques can be combined for improved performance.

### 4.4.3 Requirements

**Part A: Recommendation Approaches** Implement and compare **at least three different approaches** to generating product recommendations. The approaches should include different algorithmic strategies and feature representations.
For each approach, implement the following query:

- Given a user ID, retrieve the top $N$ recommended products.

**Part B: Cold-Start Handling** Implement and evaluate a strategy for handling the cold-start problem for both new users and new products. Demonstrate the effectiveness of your approach compared to your main recommendation methods.

**Part C: Evaluation Framework** Develop a comprehensive evaluation framework to compare all your recommendation approaches. Include appropriate baseline methods (e.g., random, popularity-based) and use suitable metrics for both rating prediction and ranking quality.

### 4.4.4 Deliverables

- **Recommendation Implementations:** Working code for all recommendation approaches.
- **Hyperparameter Tuning:** Evidence of parameter selection with justification for your final choices.
- **Query Results:** For 5 sample users, show the top-10 recommendations from each method.
- **Performance Analysis:** Comprehensive evaluation of all methods including baseline comparisons.
- **Analysis:** A discussion (400–600 words) covering:

- How do the different recommendation approaches compare in terms of accuracy and quality?
- How effective is your cold-start handling strategy?
- Which approach would you recommend for a production e-commerce system and why?

# 5 Submissions

## 5.1 Technical Report

Submit a comprehensive technical report (maximum **5,000 words**, excluding references and appendices) that documents your complete project. The report should be structured as follows:

1. **Executive Summary** (200–300 words): Overview of the project, key findings, and main recommendations.
2. **Introduction**: Project context, objectives, and scope.
3. **Data Description**: Dataset characteristics, preprocessing steps, and any data quality issues encountered.
4. **Methodology**: For each task, describe your approach, design decisions, and implementation details.
5. **Results and Analysis**: Present your findings with appropriate visualisations and critical analysis.
6. **Discussion**: Reflect on challenges encountered, limitations of your approach, and lessons learned.
7. **Conclusion and Future Work**: Summarise key outcomes and suggest improvements.
8. **References**: Cite all sources using a consistent format (e.g., IEEE, APA).
9. **Appendices**: Include supplementary material such as additional results, code snippets, or extended analyses.

The report **must include the link to your GitLab repository**.

## 5.2 Video Demonstration

Submit a **5-minute video** demonstrating your solution. The video should:

- Show your system in action (running queries, generating recommendations)
- Highlight key technical achievements

- Demonstrate that all team members understand the implementation
- Be professionally presented with clear audio and visuals

### 5.3 Presentation

Prepare presentation slides summarising your project. You will be asked to present your work to the class. Exact presentation times and scheduling will be communicated separately.

## 6 Submission Requirements

### 6.1 Deliverables

1. **Technical Report** (PDF format, maximum 5,000 words)
2. **Video Demonstration** (5 minutes maximum, MP4 format preferred)
3. **Presentation Slides** (PDF or PPTX format)
4. **GitLab Repository** (link included in report)

### 6.2 GitLab Requirements

Your GitLab repository must include:

- All source code, organised in a clear directory structure
- A comprehensive `README.md` with setup and execution instructions
- Documentation explaining how to reproduce your results
- Evidence of regular commits from all team members
- Meaningful commit messages describing changes

We will review your commit history as part of the assessment. Repositories showing contributions only in the final days before the deadline will be viewed unfavourably.

### 6.3 Submission Deadline

All deliverables must be submitted via the module portal by the deadline specified on Moodle.

# 7 Words of Wisdom

For many questions, we expect you to use your own initiative and creativity to tackle problems. You will find some parts of these questions difficult; a major hint we can give you is that you will likely face challenges with data volume and computational constraints, as well as the complexity of the raw Amazon Reviews data itself. Hence, there are a number of methods to address specific problems; we will leave it to you to interpret this. For example, it might not be wise to dive immediately into the entire dataset; instead, work on a smaller subset of a category.

This is particularly true regarding how you might approach your project and teamwork. Questions pertaining to database design, similarity computation, clustering, and recommendation systems will be decided by you and your teammates. We're not only giving you opportunities to demonstrate your technical depth, but also providing experience and reflections on managing data science projects in a team environment.

The questions are not fully isolated from one another and should instead interweave into everyone's work. For example, the similarity computation in Question 3 directly feeds into the clustering analysis in Question 4, and both inform the recommendation system in Question 5. You should also consider the project's critical path so that no one is waiting on a single team member to complete data preprocessing or feature engineering. Many of the questions depend on prior analysis, so we advise that, instead of working independently on your role, you tackle them together or in smaller pairs.

## 7.1 Tools of the Trade

You are welcome to use the programming languages and tools you feel most comfortable with (Python, PySpark, Scala, Java, R, etc.). For database components, you may choose technologies that suit your approach. There are no right or wrong tools; however, you will need to defend your choices and demonstrate why the selected tools are appropriate for the task at hand.

This doesn't give you free rein to download libraries off the Internet to automate the entire procedure for you. Please exercise good judgment, as we will be appraising precise contributions and knowledge. For example, using a pre-built recommendation library that handles the entire pipeline with a single function call would not demonstrate your understanding of the underlying concepts.

## 7.2 Better Together

We strongly encourage regular team meetings to ensure alignment and to identify dependencies early. Document your design decisions so that all team members understand the rationale behind key choices. The most successful groups will be those who communicate effectively and recognise that their individual contributions strengthen the collective output.

## 7.3 Back up Everything - Seriously!

Hopefully, you've learnt in the past few weeks how complex data science projects can be, particularly when working with large datasets and multiple team members. We strongly recommend you back up your work regularly, commit frequently to GitLab, and avoid leaving large pushes until the last minute. The worst thing that could happen is if someone accidentally overwrites critical code or loses processed results a couple of days before the deadline.

GitLab is not optional, it is a core requirement of this coursework. You must include the link to your GitLab repository in your final report, and we will thoroughly review your commit history, branching strategy, and documentation as part of the assessment. We will be looking for evidence of consistent contribution from all team members throughout the project duration, not just a flurry of activity in the final days. A well-maintained repository with regular commits, meaningful messages, and clear documentation will reflect positively on your group's professionalism and organisation.

If you can avoid it, try not to store large data files directly in your repository. Instead, document clearly in your README how to obtain and set up the data. Use .gitignore appropriately to exclude data files, temporary outputs, and system files from version control.